Experiment 6 : Maze Exploration with Q-Learning

Aim: Train an agent with reinforcement learning to find its way through a maze.

In [1]:
```
# A52 Dipanshu Ambilkar
import numpy as np
```

In [2]:
```
SIZE = 8
START, GOAL = (0, 0), (7, 7)
WALLS = {(1, 1), (1, 2), (1, 3), (2, 3), (3, 3), (4, 3), (5, 3),
         (5, 4), (5, 5), (4, 5), (3, 5), (3, 4), (6, 1), (6, 2)}
print("Maze size:", SIZE, "x", SIZE, "| walls:", len(WALLS))
```

```
Maze size: 8 x 8 | walls: 14
```

In [3]:
```
MOVES = {"up": (-1, 0), "down": (1, 0), "left": (0, -1), "right": (0, 1)}
moves = list(MOVES)
print("Moves:", moves)
```

```
Moves: ['up', 'down', 'left', 'right']
```

In [4]:
```
def step(pos, move):
    r, c = pos[0] + MOVES[move][0], pos[1] + MOVES[move][1]
    if not (0 <= r < SIZE and 0 <= c < SIZE) or (r, c) in WALLS:
        return pos, -5, False
    if (r, c) == GOAL:
        return (r, c), 20, True
    return (r, c), -1, False

print("Test: move down from start gives", step(START, "down"))
```

```
Test: move down from start gives ((1, 0), -1, False)
```

In [5]:
```
rng = np.random.default_rng(7)
Q = np.zeros((SIZE, SIZE, 4))
print("Q table shape:", Q.shape)
```

```
Q table shape: (8, 8, 4)
```

In [6]:
```
alpha, gamma, epsilon = 0.2, 0.95, 0.15
print("alpha:", alpha, "| gamma:", gamma, "| epsilon:", epsilon)
```

```
alpha: 0.2 | gamma: 0.95 | epsilon: 0.15
```

In [7]:
```
for ep in range(1500):
    pos, done = START, False
    while not done:
        r, c = pos
        if rng.random() < epsilon:
            a = rng.integers(4)
        else:
            a = int(np.argmax(Q[r, c]))
        npos, reward, done = step(pos, moves[a])
        nr, nc = npos
        Q[r, c, a] += alpha * (reward + gamma * np.max(Q[nr, nc]) - Q[r,
        pos = npos
print("Training finished: 1500 episodes")
```

```
Training finished: 1500 episodes
```

In [8]:
```
pos, path = START, [START]
while pos != GOAL and len(path) < 100:
    r, c = pos
    a = int(np.argmax(Q[r, c]))
    pos, _, _ = step(pos, moves[a])
    path.append(pos)

print("Path length:", len(path) - 1)
print("Reached goal:", path[-1] == GOAL)
```

```
Path length: 14
Reached goal: True
```

In [9]:
```
print("Path:", path)
```

```
Path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 5), (2, 5), (2, 6)
7), (3, 7), (4, 7), (5, 7), (6, 7), (7, 7)]
```

In [10]:
```
print("Best first move from start:", moves[int(np.argmax(Q[START[0], STAR
```

```
Best first move from start: right
```

In [11]:
```
grid = [["." for _ in range(SIZE)] for _ in range(SIZE)]
for (r, c) in WALLS:
    grid[r][c] = "#"
for (r, c) in path[1:-1]:
    grid[r][c] = "*"
grid[START[0]][START[1]] = "S"
grid[GOAL[0]][GOAL[1]] = "G"
print("Maze (S=start, G=goal, #=wall, *=path):")
for row in grid:
    print(" ".join(row))
```

```
Maze (S=start, G=goal, #=wall, *=path):
S * * * * * . .
. # # # . * . .
. . . # . * * *
. . . # # # . *
. . . # . # . *
. . . # # # . *
. # # . . . . *
. . . . . . . G
```